

———— **Course** ————
COMP 3609 - Introduction to Game Programming

———— **Group Members** ————
816039190 - Dominic Warrick
816039538 - Nishant Rogers

1. Game Concept

Spacewar is a two-level space shooter, built in Java using Java Swing and Java2D. The player pilots a space-ship and must destroy incoming alien enemies to accumulate points before the fuel runs out. The game escalates into a boss encounter in Level 2, where the player must fight a powerful alien boss with a multi-hit-point health system.

The game supports both single-player and two-player cooperative modes. Players select their mode from a dedicated player-select screen before the game begins.

2. How to Play

Objective: Destroy Alien enemies and Asteroids to earn points. Survive level 1 by managing your “fuel” in the form of your remaining time and evading enemy projectiles. Defeat the Boss in Level 2 to completely win the game.

Controls:

Player 1: A = Move Left | D = Move Right | W = Fire

Player 2: Left Arrow = Move Left | Right Arrow = Move Right | Up Arrow = Fire

Level 1:

- ❖ Alien enemies spawn at random intervals at the top of the screen and drift downward to a set y position, occasionally changing horizontal direction.
- ❖ Aliens fire projectiles with different movement patterns depending on their colour (see Entity Descriptions below).
- ❖ Asteroids fall from the top of the screen. Contact with an Asteroid kills the player instantly.
- ❖ Shooting an Asteroid destroys it, awards points, and drops an Energy Canister.
- ❖ Collecting an Energy Canister refills a portion of the ship's fuel, which adds 10 seconds to the remaining time.
- ❖ Level 1 lasts 90 seconds (1.5 minutes). When the timer expires, the player transitions to Level 2.

Level 2:

- ❖ A dramatic "incoming boss" audio cue plays before the Boss Alien slides onto the screen from above.
- ❖ The Boss Alien's health bar fills with an animated fill-up sequence before it becomes active. The player cannot fire until the health bar is fully displayed.
- ❖ The Boss fires all three projectile types at one-second intervals and moves horizontally across the screen.
- ❖ Asteroids continue to spawn in Level 2. Shooting it will drop a Power Gem.
- ❖ Collecting a Power Gem permanently increases the ship's projectile damage by 1.
- ❖ Defeating the Boss awards 10,000 bonus points, triggers a chain of explosion animations, plays a victory audio track, and transitions to the Win Screen.

Screens:

- ❖ Start Screen: Main menu with Start, Instructions, and Exit buttons, plus volume control buttons.
- ❖ Instructions: In-game overlay describing controls and objectives.
- ❖ Player Select: Choose 1 Player or 2 Players before starting.
- ❖ Pause Screen: Accessible via the pause button (top-left). Options to Resume, Restart, or Exit, with volume controls.
- ❖ Game Over Screen: Displays current score and all-time high score, with options to Start Over or Exit.
- ❖ Win Screen: Displays gold "YOU WIN!" text, current score, high score, and options to Play Again or Exit.

3. Entity Descriptions

Player Ship (Ship.java)

The player's space-ship, rendered using an animated sprite sheet. Three animations exist: idle, moving left, and moving right, which are swapped depending on the player's movement input to create the illusion of thrusters moving them horizontally.

Each ship has a configurable damage value that increases when a Power Gem is collected. A fire cooldown of 300 ms prevents the player from holding the fire key to spam bullets. In two-player mode, a second Ship instance (Player 2) is created with its own distinct sprite sheet and keyboard bindings.

Alien (Alien.java)

A standard alien enemy that spawns at the top of the screen and drifts downward.

There are three variants distinguished by a colour tint applied via the ImageFX system:

- ❖ No tint (Green): Fires a straight-line projectile downward.
- ❖ Blue tint: Fires a sine-wave projectile.
- ❖ Red tint: Fires a circular-motion projectile.

Each Alien fires once every two seconds. Their horizontal movement changes randomly each frame (5% chance per frame) to give unpredictable horizontal drift, clamped to the screen boundaries.

Boss Alien (Boss.java)

A large, uniquely animated enemy that slides in from above at the start of Level 2. The Boss has 500 hit points in single-player mode and 1000 in two-player mode. It uses all three projectile firing modes (straight, sine, and circular), selecting randomly each time it fires. A health bar rendered above the Boss fills up with an animated sequence before the Boss becomes active. A damage flash effect (red glow timer) triggers visually whenever the Boss takes a hit.

Asteroid (Asteroid.java)

A falling obstacle drawn entirely with Java2D geometric shapes and gradient paint. It is rendered as an ellipse with crater details, surface texture, and a red outline stroke. Asteroids fall at a fixed downward speed. When shot, the Asteroid is destroyed, awarding points and dropping an item. If an Asteroid contacts the player's ship, the ship is destroyed.

Item Interface (Item.java)

A Java interface defining the blueprint for all collectible items: `update()`, `draw()`, `getBoundingBox()`, `isActive()`, `getX()`, and `getY()`. Both `EnergyCanister` and `PowerGem` implement this interface, enabling uniform handling in the game loop.

Energy Canister (EnergyCanister.java)

A Level 1 item that drops from destroyed Asteroids. It falls downward using an animated sprite sheet (`sodaCan.png`). A `GlowFX` image effect renders a golden aura around it to make it visually distinct. Collecting it refills ship fuel.

Power Gem (PowerGem.java)

A Level 2 item that drops from destroyed Asteroids. Rendered as a static image with the same `GlowFX` golden aura effect as the `Energy Canister`. Collecting it permanently increments the player ship's damage value, increasing projectile damage output.

Projectile Interface (Projectile.java)

A Java interface that defines the blueprint for all projectile types: `update()`, `draw()`, `getBoundingBox()`, `isActive()`, and `isShip()`. This allows the game loop to treat all projectile variants uniformly in collections.

StraightProjectile (StraightProjectile.java)

A simple 8x8 pixel rectangle that travels in a straight vertical line. It moves upward when fired by the player and downward when fired by an Alien or Boss. Projectile colour is configurable; player colours are determined by ship colour index.

SineProjectile (SineProjectile.java)

A projectile whose horizontal position oscillates following a sine wave as it travels vertically. The amplitude is 75 pixels and the frequency factor is 1. The sine calculation is: $x = \sin((2y / 180.0) * \text{PI}) * \text{amplitude}$, centered on the spawn x-coordinate.

CircularProjectile (CircularProjectile.java)

A projectile that traces a circular arc as it descends. The center of the circle moves downward at a fixed speed, and the projectile orbits the center using the parametric circle equations: $x = \text{centreX} + \text{radius} * \cos(\text{angle})$, $y = \text{centreY} + \text{offset} - \text{radius} * \sin(\text{angle})$. The angle increments by 9 degrees per frame.

4. Class Descriptions and Object-Oriented Model Diagram

The project is organized into four packages plus a top-level application main class:

GameApplication - Main class. Instantiates GameWindow.

GameWindow - Extends JFrame. Creates and sizes the window, adds GamePanel, and starts the game thread.

GamePanel - Extends JPanel, implements Runnable, KeyListener, MouseListener. Central game controller: owns the game loop, manages all entity collections, handles input, collision detection, level transitions, scoring, and rendering.

———— Package: GameEntities ————

Projectile - Interface for all projectile objects.

Item - Interface for all collectible item objects.

Ship - Player-controlled spaceship entity with animation, movement, and firing logic.

Alien - Enemy entity with tint-based variants and projectile firing delegation.

Boss - Boss enemy with animated health bar, damage flash effect, and multi-mode firing.

Asteroid - Java2D-drawn falling obstacle.

StraightProjectile - Implements Projectile; vertical linear path.

SineProjectile - Implements Projectile; sinusoidal path.

CircularProjectile - Implements Projectile; circular orbit path.

EnergyCanister - Implements Item; animated fuel pickup / time increase.

PowerGem - Implements Item; static damage-boost pickup.

———— Package: ImageManip ————

ImageFX - Interface for image post-processing effects (apply, update, getEffectName).

TintFX - Implements ImageFX; applies a colour channel boost (red, blue, or green) to every pixel of a BufferedImage to tint alien sprites.

GlowFX - Implements ImageFX; recolours non-transparent pixels to a specified RGB with contrast boost, used to create golden aura effects on items.

ImageManager - Static utility class for loading Image and BufferedImage objects from file paths, and for deep-copying BufferedImage instances.

Package: Misc

Animation - Loads a sprite sheet, slices it into frames by grid dimensions, and cycles through them at a configured frame duration.

SoundManager - Singleton that pre-loads all audio clips at startup using javax.sound.sampled. Provides play, stop, loop, pause, resume, and volume-control methods. Tracks currently looping and paused clips.

Package: Screens

ParallaxLayer - Wraps a BufferedImage and a scroll speed factor. Advances the image's y-offset each frame and wraps it to create infinite scrolling.

BackgroundManager - Has three ParallaxLayer instances (space backdrop, planets, and stars) and draws them each frame with movement paused when the game is paused.

Explosion - Wraps an Animation of an explosion sprite sheet (non-looping, 5x4 frames). Active until the animation completes.

StartScreen - Draws the main menu: game title, Start, Instructions, and Exit buttons, a volume control row, and an in-overlay instructions panel.

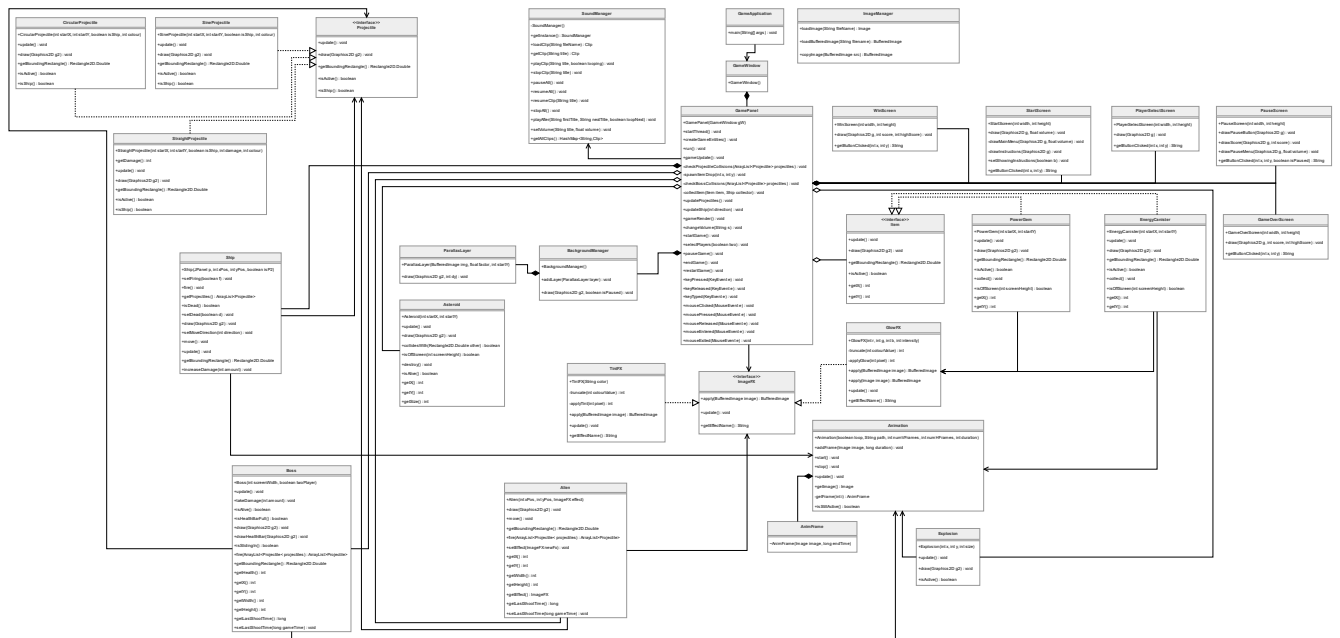
PlayerSelectScreen - Draws the 1 Player / 2 Players selection screen.

PauseScreen - Draws the in-game pause button (top-left corner), the live score display, the dimmed pause overlay with Resume / Restart / Exit buttons, and volume controls.

GameOverScreen - Draws the Game Over overlay with the current score, all-time high score, and Start Over / Exit buttons.

WinScreen - Draws the You Win overlay with gold title text, the current score, the high score, and Play Again / Exit buttons.

Object Oriented Model: Implemented as an SVG, you can zoom into the pdf to view specifics.



Zoomable, interactive diagram via google drive:

https://drive.google.com/file/d/1u4REExQpdcD9AbO5Wks2C2qvsis_XyyP/view?usp=sharing

5. Sources of Images, Sound Clips and Code

Images:

images/shipIdleP1.png	—	Sprite sheet for Player 1 ship idle animation
images/rightP1.png	—	Sprite sheet for Player 1 ship right-lean animation
images/leftP1.png	—	Sprite sheet for Player 1 ship left-lean animation
images/shipIdleP2.png	—	Sprite sheet for Player 2 ship idle animation
images/rightP2.png	—	Sprite sheet for Player 2 ship right-lean animation
images/leftP2.png	—	Sprite sheet for Player 2 ship left-lean animation
images/Alien.png	—	Base alien sprite (tinted at runtime for variants)
images/BossAlien.png	—	Animated boss sprite sheet (6 columns x 3 rows)
images/sodaCan.png	—	Animated Energy Canister sprite sheet (4 cols x 3 rows)
images/powerGem.png	—	Static Power Gem item sprite
images/explosion.png	—	Explosion animation sprite sheet (4 cols x 5 rows)
images/Space.png	—	Parallax background - deep space layer
images/Planets.png	—	Parallax background - mid-distance planets layer
images/Stars.png	—	Parallax background - near-distance stars layer

Asteroid visuals and projectile shapes (squares) are drawn procedurally using Java2D (Graphics2D, GradientPaint, Ellipse2D, BasicStroke) with no external image files.

Custom sprites (ships, aliens, boss, energy canister, power gem) were created specifically for this project. All images and sprites, with the exception of the explosion sprite, were created by Dominic Warrick.

Explosion sprite source:

https://www.pngitem.com/middle/hToJxb_preview-pixel-art-explosion-sprite-sheet-hd-png/

Sound Clips:

sounds/mainmenu.wav

Background music for the main menu

Source: <https://www.youtube.com/watch?v=ooSEwO5I5SE>

sounds/bgm.wav

Background music for Level 1 gameplay

Source: <https://www.youtube.com/watch?v=hPEQLU-aD3I>

sounds/boss.wav

Background music for the Boss encounter (Level 2)

Source: https://www.youtube.com/watch?v=opvuEVpl_PM

sounds/startofboss.wav

Audio sting that plays before the boss music starts

Source: https://www.youtube.com/watch?v=opvuEVpl_PM

sounds/explosion.wav

Sound effect played on Asteroid/Boss destruction

Source: <https://www.youtube.com/watch?v=9FMquJzgDGQ>

sounds/canister.wav

Sound effect played when an Energy Canister is collected

Source: https://www.youtube.com/watch?v=IRMToua9D_0

sounds/gem.wav

Sound effect played when a Power Gem is collected

Source: <https://www.youtube.com/watch?v=GjdTJF6Wo-I>

sounds/start.wav

Sound effect played when the game starts

Source: ElevenLabs audio

sounds/death.wav

Sound effect played on Game Over

Source: ElevenLabs audio

sounds/winning.wav

Sound effect/music played on victory (boss defeated)

Source: ElevenLabs audio

All audio files are in WAV format and loaded via javax.sound.sampled.

All the walkie-talkie audios were generated via ElevenLabs TTS.

Libraries:

Java Standard Library:

- ❖ javax.swing (JFrame, JPanel, ImageIcon)
- ❖ java.awt and java.awt.image (Graphics2D, BufferedImage, GradientPaint, BasicStroke, Ellipse2D, Rectangle2D, Color, Font, etc.)
- ❖ javax.sound.sampled (AudioSystem, Clip, AudioInputStream)
- ❖ java.util (ArrayList, HashMap, HashSet, Random)
- ❖ java.io (File, IOException)

All game logic, rendering, animation, image effects, sound management, entity behaviour, and screen management code was written by the project team. No third-party game frameworks or external libraries were used.

———— Account of work done by group members ————

Nishant Rogers: All sound clip discovery and editing. All menu and game UI / screens and implementation. Asteroid and corresponding Item logic. Level logic. Spaceship firing logic. Co-op implementation. Boss start animation. Pausing and Resuming logic, inclusive of clips. GlowFX implementation.

Dominic Warrick: All sprite work, as previously mentioned. All parallax backgrounds. Projectile and movement implementation for aliens and boss, along with the math for them. Space-ship movement logic. Volume logic. TintFX implementation.

Both members: Requirements Analysis document, Design document, Final report document. Game balancing. Game Demo.

Signed off by:

Dominic Warrick

Nishant Rogers

———— Demonstration Video ————

[\[https://youtu.be/NDrCBEu5MkM\]](https://youtu.be/NDrCBEu5MkM)

———— Repository Link ————

[\[https://github.com/FishyFoshy/GameProgramming-Project\]](https://github.com/FishyFoshy/GameProgramming-Project)